Advanced Quantitative Research Methods

Support Vector Machines

Advanced module A5 — optional self-study

Prof. Dr. Christoph Weisser

Slides, lab notebooks and data: 

github.com/ChrisW09/Advanced-Quantitative-Research-Methods

The advanced modules at a glance

Module	Topic
A1	Randomised Controlled Trials — potential outcomes, randomisation, power
A2	Explainable AI with Shapley Values — axioms, exact and Monte-Carlo Shapley
A3	Conformal Prediction — split conformal, CQR, prediction sets
A4	GLMs and Splines — exponential family, overdispersion, P-splines
A5	**Support Vector Machines** — margins, the cost C, kernels
A6	Survival Analysis — censoring, Kaplan–Meier, Cox
A7	Multiple Testing — FWER, Bonferroni and Holm, FDR
A8	Unsupervised Learning — PCA, K-means, hierarchical clustering
A9	Deep Learning — MLPs, convolutions, training, regularisation

Twelve optional, self-study modules sit outside the taught chapter plan (the table shows A1–A9; A10–A12 are newer additions — see the folder README); today's module is highlighted. Each is a full deck in the course house style with a companion lab notebook.

This module assumes Chapter 4 (classification, the confusion matrix, ROC/AUC) and Chapter 5 (cross-validation). It was ISLP Chapter 9 in earlier editions of this course.

Contents

1. Why This Chapter
2. Hyperplanes
3. Maximal Margin
4. Soft Margin
5. Kernels
6. Many Classes
7. Tuning and Comparison
8. Python Lab
9. Summary

Optional and advanced material is collected in the **appendix** at the end of the deck.

Notation in this chapter

Symbol	Meaning
$x_i \in \mathbb{R}^p$	predictor vector of observation i; n observations, p predictors
$y_i \in \{-1, +1\}$	class label, coded ± 1 — the sign carries the class
β_0, $\beta = (\beta_1, \ldots, \beta_p)^\top$	intercept and coefficient vector of the hyperplane
$f(x) = \beta_0 + \beta^\top x$	the score; the decision rule is $\hat{y} = \operatorname{sign} f(x)$
$\|\beta\| = \sqrt{\sum_j \beta_j^2}$	Euclidean norm of the coefficient vector
M	the margin: distance from the hyperplane to the nearest point
$y_i f(x_i)$	functional margin of observation i
$\xi_i \geq 0$	slack of observation i (ISLP writes ϵ_i)
C	sklearn's cost: *inverse* regularisation (large C = narrow margin)
C_{ISLP}	ISLP's budget: the cap on $\sum_i \xi_i$ (large budget = wide margin)
λ	ridge-style penalty in the hinge-loss form, $\lambda = 1/(2C)$
$\mathcal{S}$, α_i	support-vector index set; dual coefficients ($\alpha_i \neq 0$ only on $\mathcal{S}$)
$K(x, x')$	kernel: a similarity between two observations
d, γ	polynomial degree; radial-kernel scale
$\phi(\cdot)$, K	implicit feature map; number of classes (multi-class section only)

Sources and references

Primary textbook

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning, with Applications in Python.* Springer.

Learning objectives

By the end of this chapter you will be able to:

- **Explain** a separating hyperplane, the margin and why only the support vectors matter.
- **State** the maximal-margin and soft-margin optimisation problems, including the budget on the *sum* of the slacks.
- **Use** the kernel trick to fit non-linear boundaries with polynomial and radial kernels.
- **Tune** C and γ by cross-validation, and read the bias–variance trade-off off the result.
- **Avoid** the two traps that ruin SVM work: the inverted C convention and unscaled predictors.
- **Compare** SVMs with logistic regression and with trees and forests, on losses and on ROC curves.

Roadmap of this chapter

Classification by geometry

1. Separating hyperplanes: a classifier that is a *boundary*.
2. The maximal margin classifier — and its two fatal flaws.
3. The support vector classifier: slacks and the budget.
4. Support vector machines: kernels, degree d and γ.
5. More than two classes: one-vs-one, one-vs-all.
6. Tuning by cross-validation, and honest comparisons.

Three names, three nested models

Maximal margin classifier $\subset$ *support vector classifier* (soft margin, linear) $\subset$ *support vector machine* (soft margin, kernel).

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

A classifier defined by a boundary, not a probability

Chapter 4 modelled $\Pr(Y = 1 \mid X)$ and then cut it at 0.5. Chapter 9 reverses the order: it draws the **boundary first** and never models a probability at all.

Two different questions

- **Logistic regression:** which coefficients make the observed labels most likely? (Maximum likelihood — every point contributes.)
- **The SVM:** where is the *widest street* that keeps the classes apart? (A geometric optimum — only the points at the kerb contribute.)

Takeaway

Same data, same ± 1 labels, different objective — so the two will disagree. Section 7 pins down exactly where: they differ only in their *loss function*.

Why maximum likelihood can fail where the margin does not

Perfect separation breaks maximum likelihood

If a hyperplane separates the classes perfectly, scaling (β_0, β) by $t > 1$ scales every score $f(x_i)$ by t, pushes every fitted probability closer to 0 or 1, and so *increases* the log-likelihood. The maximum is reached only as $\|\beta\| \to \infty$: the MLE does not exist.

Takeaway

Maximising the margin is the opposite instinct: it asks for the *smallest* $\|\beta\|$ compatible with correct classification, because the margin is $M = 1/\|\beta\|$. On separable data that is always a finite, unique answer — which is why SVMs were first adopted in earnest on gene-expression data, where p in the thousands and n in the dozens make the classes *always* separable.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

Hyperplanes, and the rule they define

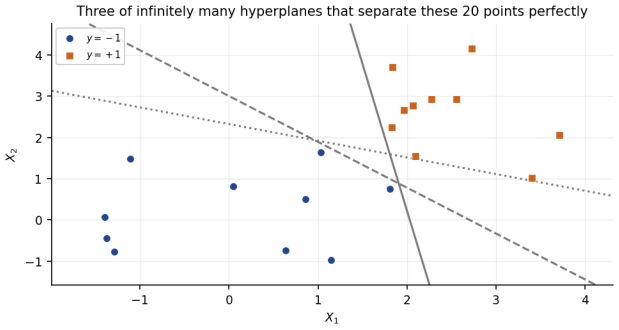
Definition and decision rule

$$\{x \in \mathbb{R}^p : \beta_0 + \beta^\top x = 0\}, \qquad \hat{y}(x) = \operatorname{sign} f(x), \quad f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

How to read this

- β is *perpendicular* to the plane and fixes its orientation; β_0 fixes the offset. In $p = 2$ it is a line, in $p = 3$ a plane, in general a flat $(p-1)$-dimensional sheet.
- Coding $y_i = \pm 1$ makes "correct" one inequality for both classes: $y_i f(x_i) > 0$. This is the **functional margin**.
- The signed distance from x_0 to the plane is $f(x_0)/\|\beta\|$ — the score, rescaled. So $|f|$ is a confidence, **not** a probability.
- In scikit-learn the score is `decision_function(X)`; `predict` is just its sign.

If the classes separate, there are infinitely many answers



Takeaway

All three lines have zero training error on these 20 simulated points (seed 2024), and so do infinitely many others. **Zero training error does not pick a model** — we need a second criterion.

Exercise 9.1 — Reading a hyperplane [Math]

Exercise

Consider the hyperplane $1 + 2X_1 + 3X_2 = 0$ in $\mathbb{R}^2$ and the rule $\hat{y} = \text{sign}(1 + 2X_1 + 3X_2)$.

1. Classify the points $(0, 0)$, $(-1, 1)$ and $(2, -2)$.
2. Compute $\|\beta\|$ and the perpendicular distance from each point to the hyperplane.
3. All three coefficients are multiplied by 10. What happens to the predicted classes, and what to the scores $f(x)$?
4. Which is invariant to that rescaling: the sign of f, or its size?

Solution — Exercise 9.1 (1/2)

Solution

Method. Evaluate $f(x) = 1 + 2x_1 + 3x_2$, read the class off its sign, and divide $|f|$ by $\|\beta\|$ to turn the score into a distance.

Parts 1–2. $\|\beta\| = \sqrt{2^2 + 3^2} = \sqrt{13} = 3.6056$.

x	$f(x)$	$\hat{y}$	$\lvert f(x)\rvert / \|\beta\|$
$(0, 0)$	$1 + 0 + 0 = +1$	$+1$	$1/3.6056 = 0.277$
$(-1, 1)$	$1 - 2 + 3 = +2$	$+1$	$2/3.6056 = 0.555$
$(2, -2)$	$1 + 4 - 6 = -1$	-1	$1/3.6056 = 0.277$

Solution — Exercise 9.1 (2/2)

Solution

So $(0,0)$ and $(-1,1)$ are on the positive side, $(2,-2)$ on the negative side, and $(-1,1)$ is twice as far from the boundary as the other two.

Parts 3–4. Multiplying by 10 gives $f \to 10f$ *and* $\|\beta\| \to 10\|\beta\|$: every score is ten times larger, but every sign — hence every prediction — is unchanged, and so is $|f|/\|\beta\|$. **Sign and distance are invariant; the raw score is not.**

Common mistake

Reading $|f(x)|$ as a distance without dividing by $\|\beta\|$. The scale of f is arbitrary — which is why the next section *fixes* it by demanding $y_i f(x_i) \geq 1$ at the support vectors.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

The margin: the widest street between the classes

Among all separating hyperplanes, take the one whose nearest training point is as far away as possible.

The maximal margin problem

$$\max_{\beta_0,\,\beta,\,M} M \quad \text{s.t.} \quad \|\beta\| = 1, \qquad y_i\left(\beta_0 + \beta^\top x_i\right) \geq M \quad \forall\, i = 1, \ldots, n$$

How to read this

- $M > 0$ is the **margin**: the perpendicular distance from the hyperplane to the closest observation. The street has total width $2M$.
- The constraint $\|\beta\| = 1$ removes the arbitrary scale of Exercise 9.1, so $y_i(\beta_0 + \beta^\top x_i)$ *is* a signed distance.
- The n inequalities say: every point is on the correct side *and* at least M away.

The same problem, in the form software actually solves

Equivalent convex form: drop $\|\beta\| = 1$, fix the scale instead

$$\min_{\beta_0, \beta} \ \tfrac{1}{2}\|\beta\|^2 \quad \text{s.t.} \quad y_i\left(\beta_0 + \beta^\top x_i\right) \geq 1 \ \ \forall\, i, \qquad \text{then} \quad M = \frac{1}{\|\beta\|}$$

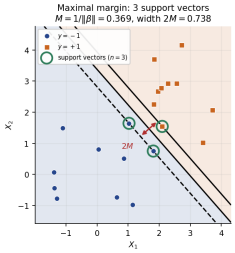
Why the two are the same problem

- Rescale (β_0, β) so the closest points have $y_i f(x_i) = 1$ exactly — the *canonical* scaling.
- With that scaling the geometric margin is $1/\|\beta\|$, so *maximising* M is *minimising* $\|\beta\|$.
- Minimising $\frac{1}{2}\|\beta\|^2$ under linear constraints is a convex quadratic programme with a unique solution.

Takeaway

"Widest margin" = "smallest coefficients". This is already a ridge-type shrinkage of β (Chapter 6); Section 7 makes the penalty explicit.

Support vectors: the only points that matter



Takeaway

On the seeded data ($n = 20$, seed 2024), $\|\beta\| = 2.711$, so $M = 1/\|\beta\| = 0.369$ and the street is $2M = 0.738$ wide. Exactly **three** points touch the kerb ($y_i f(x_i) = 1$): the **support vectors**. Delete any of the other 17 and the fit is identical — they carry $\alpha_i = 0$ in $f(x) = \beta_0 + \sum_i \alpha_i \langle x, x_i \rangle$.

History: Vladimir Vapnik's idea from the 1960s; kernels and soft margins made it the SVM in the 1990s.

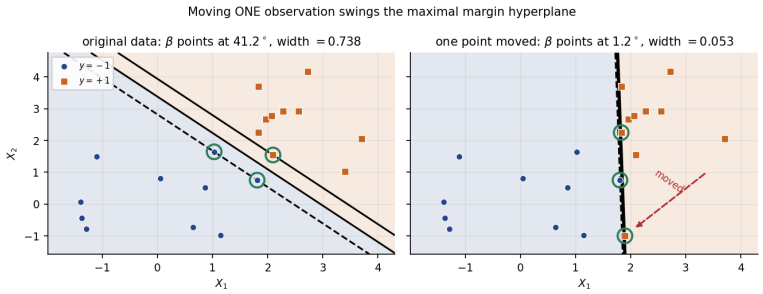
Fatal flaw 1: with overlapping classes there is no solution



Takeaway

On overlapping data ($n = 60$, seed 2025) *no* hyperplane separates the classes, so the constraint set $y_i f(x_i) \geq M > 0$ is **empty** and the problem is infeasible. Even the best linear rule misclassifies 8 of 60 points (training accuracy 0.867).

Fatal flaw 2: one observation dictates the answer



Takeaway

Move a *single* point from (3.40, 1.02) to (1.90, −1.00) — the classes still separate — and β swings from 41.2° to 1.2° while the street collapses from 0.738 to 0.053, fourteen times narrower. The support vector set changes entirely. That is **variance**, in the Chapter 2 sense.

Exercise 9.2 — The maximal margin by hand [Math]

Exercise

Seven observations in $p = 2$, with $y = -1$ and $y = +1$ as shown:

i	1	2	3	4	5	6	7
X_1	3	2	4	1	2	4	4
X_2	4	2	4	4	1	3	1
y	-1	-1	-1	-1	$+1$	$+1$	$+1$

1. Sketch the points and argue that the optimal separating line is $X_2 = X_1 - \frac{1}{2}$.
2. Write it in canonical form ($y_i f(x_i) \geq 1$, with equality at the closest points) and give β_0, β, $\|\beta\|$.
3. Compute the margin M and the street width $2M$.
4. Which observations are the support vectors? What changes if observation 7 moves to (4.5, 0.5)?

Solution — Exercise 9.2 (1/2)

Solution

Method. Find the two closest opposite-class pairs, put the boundary half-way between them, then rescale so the nearest points score exactly ± 1.

Part 1. Along the direction $X_1 - X_2$ the classes separate: the $y = -1$ points give $X_1 - X_2 \in \{-1, 0, 0, -3\}$, the $y = +1$ points give $\{1, 1, 3\}$. The extremes are 0 (points 2, 3) and 1 (points 5, 6), so the widest street puts the boundary at the midpoint $X_1 - X_2 = \frac{1}{2}$, i.e. $X_2 = X_1 - \frac{1}{2}$.

Part 2. With $g(x) = -\frac{1}{2} + x_1 - x_2$ we get $g = \mp\frac{1}{2}$ at the four closest points, so multiply by 2:

$$f(x) = -1 + 2x_1 - 2x_2, \quad \beta_0 = -1, \quad \beta = (2, -2)^\top, \quad \|\beta\| = 2\sqrt{2} = 2.8284.$$

Check: $f(2, 2) = -1$, $f(4, 4) = -1$, $f(2, 1) = +1$, $f(4, 3) = +1$, so $y_i f(x_i) = 1$ there, while $f(3, 4) = -3$, $f(1, 4) = -7$, $f(4, 1) = +5$ give $y_i f(x_i) > 1$.

Solution — Exercise 9.2 (2/2)

Solution

Part 3. $M = 1/\|\beta\| = 1/(2\sqrt{2}) = 0.3536$ and $2M = 1/\sqrt{2} = 0.7071$.
Part 4. The support vectors are the four points with $y_i f(x_i) = 1$: observations $2\,(2,2)$ and $3\,(4,4)$ from the -1 class, $5\,(2,1)$ and $6\,(4,3)$ from the $+1$ class. Observations 1, 4, 7 are strictly outside the street.
Moving observation 7 from $(4,1)$ to $(4.5, 0.5)$ takes it *further* away (f goes from $+5$ to $+7$), so it is still not a support vector: the hyperplane, the margin and the fit are **completely unchanged**.
Note for the lab: `SVC(kernel='linear', C=1e6)` recovers $\beta \approx (2,-2)$, $\beta_0 \approx -1$, but may list only *three* entries in `support_`: with four points tied on the margin the dual solution is not unique and one multiplier can be exactly zero. All four are still support vectors geometrically.

Common mistake

Putting the boundary through the midpoint of the two class *centroids*. The margin is set by the *closest* pair; interior points are irrelevant.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

Buying robustness with violations

Both flaws have one cure: stop insisting that every point be correct, and *pay* for the exceptions.

The support vector classifier (ISLP form)

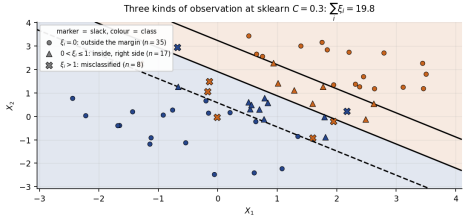
$$\max_{\beta_0,\beta,\xi,M} M \quad \text{s.t.} \quad \|\beta\| = 1, \quad y_i f(x_i) \geq M(1-\xi_i), \quad \xi_i \geq 0, \quad \sum_{i=1}^{n} \xi_i \leq C_{\text{ISLP}}$$

Read every symbol

- ξ_i is the **slack** of observation i, measured in units of the margin M.
- M is still *maximised*; what is *budgeted* is the **sum of the slacks**, $\sum_i \xi_i \leq C_{\text{ISLP}}$ — not the number of violations, and not each ξ_i separately.
- $C_{\text{ISLP}} = 0$ forces every $\xi_i = 0$: the hard margin is the special case. Because it is a *total* budget, one badly placed point can eat the whole allowance alone.

What each slack value means

Slack	Where the point sits	Support vector?
$\xi_i = 0,\ y_i f(x_i) > 1$	strictly outside the street	no — irrelevant to the fit
$\xi_i = 0,\ y_i f(x_i) = 1$	exactly on the kerb	yes
$0 < \xi_i \leq 1$	inside the street, still on its own side	yes
$\xi_i > 1$	on the *wrong* side — misclassified	yes



Takeaway

Marker shape is the slack category, colour the class. At sklearn $C = 0.3$: 35 points outside the street, 17 inside it on their own side, 8 misclassified, total $\sum_i \xi_i = 19.8$.

The form scikit-learn solves — and the trap

Hinge-loss form (this is what SVC minimises)

$$\min_{\beta_0,\beta,\xi} \ \tfrac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i f(x_i) \geq 1-\xi_i, \quad \xi_i \geq 0$$

Where the inversion comes from

Divide by C: the objective is $\frac{1}{2C}\|\beta\|^2 + \sum_i \xi_i$, so the ridge penalty weight is $\lambda = 1/(2C)$. **Large** $C \Rightarrow$ violations expensive $\Rightarrow$ $\|\beta\|$ free to grow $\Rightarrow$ **narrow** margin, *less* regularisation. **Small** $C \Rightarrow$ the reverse.

The single most common source of confusion in this chapter

ISLP's C_{ISLP} is a *budget for violations*: large budget = wide margin. Scikit-learn's C is a *cost per violation*: large C = **narrow** margin. The conventions run in **opposite** directions, roughly $C \sim 1/C_{\text{ISLP}}$. Get it backwards and your whole bias–variance story inverts.

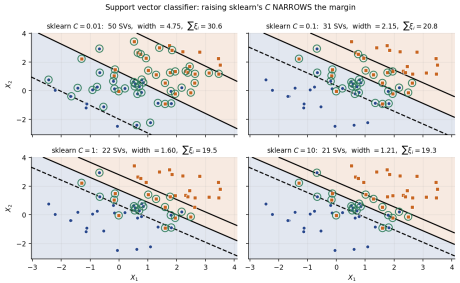
The two conventions side by side

	ISLP C_{ISLP} (a budget)	**sklearn** C (a cost)
Appears in	$\sum_i \xi_i \le C_{\text{ISLP}}$	$+ C \sum_i \xi_i$ in the objective
Larger value means	more violations *allowed*	each violation *more expensive*
Margin	wider	narrower
Support vectors	more	fewer
Flexibility	lower (more bias)	higher (more variance)
Hard margin at	$C_{\text{ISLP}} = 0$	$C \to \infty$

Takeaway

Whenever you read "increase C", check *whose* C. Every slide here that says "sklearn C", and every line of notebook code, uses the scikit-learn convention: **big C, narrow street**. Write the library and version next to any reported value — "$C = 100$, a highly regularised model" is the sentence that gets a validation report sent back.

C is the bias–variance dial (bridge to Chapter 2)



Takeaway

As sklearn C rises 0.01 $\rightarrow$ 10 on the same 60 points: street width falls 4.75 $\rightarrow$ 1.21, support vectors fall 50 $\rightarrow$ 21, $\sum_i \xi_i$ falls 30.6 $\rightarrow$ 19.3. A wide margin rests on many points, so no single one can move it — low variance, high bias. Training accuracy is not even monotone (0.850, 0.867, 0.867, 0.850): tune C by **cross-validation**.

Exercise 9.3 — Slacks, budgets and sklearn's C [Concept]

Exercise

1. State precisely what is maximised and what is constrained in the support vector classifier. Which quantity carries the budget?
2. An observation has $\xi_i = 0.4$. Where is it, and is it a support vector? Repeat for $\xi_i = 1.8$.
3. A colleague reports: "I raised `SVC(C=...)` from 0.1 to 100 to regularise the model more strongly." What is wrong, and what actually happened to the margin and to $|\mathcal{S}|$?
4. You fit a support vector classifier to $n = 200$ points and find 196 support vectors. What does that say about C and about the bias–variance position of the fit?

Solution — Exercise 9.3

Solution

Part 1. The *margin* M is maximised, under $y_i f(x_i) \geq M(1-\xi_i)$ with $\xi_i \geq 0$; the budget sits on the **sum** of the slacks, $\sum_{i=1}^{n} \xi_i \leq C_{\text{ISLP}}$ — not on each ξ_i, and not on a count of violations.
Part 2. $\xi_i = 0.4 \in (0, 1]$: inside the street but still on the correct side. $\xi_i = 1.8 > 1$: on the wrong side, *misclassified*. Both have $y_i f(x_i) \leq 1$, so **both are support vectors**.
Part 3. The direction is inverted. In scikit-learn C is the *cost per unit slack* in $\frac{1}{2}\|\beta\|^2 + C\sum_i \xi_i$, i.e. penalty weight $\lambda = 1/(2C)$. Raising C from 0.1 to 100 **weakens** regularisation: slacks shrink, $\|\hat{\beta}\|$ grows, the margin $1/\|\beta\|$ *narrows*, $|\mathcal{S}|$ *falls*. The model became *more* flexible — the opposite of the stated intent.
Part 4. 196 of 200 points have $y_i f(x_i) \leq 1$, so nearly the whole sample lies inside a very wide street: a **small** C (large ISLP budget), heavily regularised, high bias, low variance. If CV accuracy is poor, raise C; if it is already good, the wide margin is buying stability for free.

Common mistake

Treating many support vectors as evidence of overfitting. It is the opposite signal: many support vectors means a wide margin, hence a smooth, heavily regularised fit.

Extended Exercise 9.1 — From hard to soft margin by hand [Math]

Extended exercise (15 min)

Take the seven points of Exercise 9.2 with the hard-margin solution $f(x) = -1 + 2x_1 - 2x_2$, $\|\beta\| = 2\sqrt{2}$, $M = 0.3536$.

1. Add an eighth observation at $(2, 3)$ with $y_8 = +1$. Show that the data are no longer separable, so the hard-margin problem is infeasible.
2. Keeping the *same* f, compute $\xi_i = \max\{0,\, 1 - y_i f(x_i)\}$ for all eight points and the total $\sum_i \xi_i$.
3. How large must C_{ISLP} be for this f to be feasible? Which observations are now support vectors?
4. In terms of $\frac{1}{2}\|\beta\|^2 + C\sum_i \xi_i$, what would a *small* sklearn C do to this fit instead?

Solution — Extended Exercise 9.1 (1/2)

Solution — parts 1 and 2: separability, and the slacks

Part 1. The new point $(2, 3)$ has $x_1 - x_2 = -1$, the same value as observation 1 at $(3, 4)$ — but observation 1 has $y = -1$ and the new point $y = +1$. Along that direction the classes now overlap; in fact the convex hulls of the two classes intersect. Two intersecting convex sets cannot be separated by a hyperplane, so no (β_0, β) satisfies $y_i f(x_i) \geq M > 0$ for all i: the programme is **infeasible**.
Part 2. With $f(x) = -1 + 2x_1 - 2x_2$:

i	1	2	3	4	5	6	7	8
x	$(3,4)$	$(2,2)$	$(4,4)$	$(1,4)$	$(2,1)$	$(4,3)$	$(4,1)$	$(2,3)$
y_i	-1	-1	-1	-1	$+1$	$+1$	$+1$	$+1$
$f(x_i)$	-3	-1	-1	-7	$+1$	$+1$	$+5$	-3
$y_i f(x_i)$	3	1	1	7	1	1	5	-3
ξ_i	0	0	0	0	0	0	0	4

Only the new point violates anything, and it violates a lot: $\xi_8 = 1 - (-3) = 4$, so $\sum_{i=1}^{8} \xi_i = 4$. **One badly placed observation consumes the entire budget** — the honest reading of "the budget is on the *sum* of the slacks".

Solution — Extended Exercise 9.1 (2/2)

Solution — parts 3 and 4: budget, support vectors, sklearn's C

Part 3. This f becomes feasible as soon as $C_{\text{ISLP}} \geq \sum_i \xi_i = 4$. The support vectors are every point with $y_i f(x_i) \leq 1$: observations $2, 3, 5, 6$ (on the kerb, $\xi = 0$) and observation 8 (misclassified, $\xi_8 = 4$) — five in all. Observations 1, 4, 7 remain irrelevant.
Part 4. In the scikit-learn objective this f costs $\frac{1}{2}(2\sqrt{2})^2 + 4C = 4 + 4C$. With a *small* C the $4C$ term is cheap, so the solver buys more slack in exchange for a smaller $\|\beta\|$: it tilts and *widens* the street, lets several points drift inside, and becomes smoother and far less sensitive to point 8. With a *large* C it contorts to keep ξ_8 small, ending up with a large $\|\beta\|$ and a narrow, unstable margin.

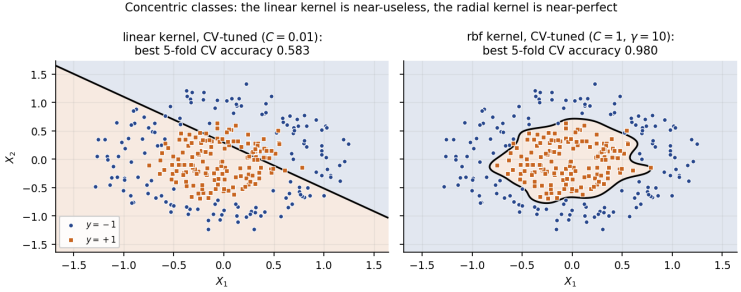
Common mistake

Computing ξ_8 as a geometric distance. The slack lives on the *functional* margin scale, $\xi_i = \max\{0, 1 - y_i f(x_i)\}$, so it is 4 here — not $4/\|\beta\| = 1.41$.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

When no straight line will do



Takeaway

300 points on two concentric rings (`make_circles`, `random_state=0`). Even with C chosen by 5-fold CV the **linear** kernel reaches only 0.583 accuracy — barely above the 0.500 coin flip — while the radial kernel reaches 0.980. No amount of soft-margin tuning fixes a wrong *shape*.

The old trick: enlarge the feature space

Chapter 7's answer: add non-linear columns and fit a *linear* rule in the bigger space.

The concentric data, made separable

Add one feature, $X_3 = X_1^2 + X_2^2$. In (X_1, X_2, X_3)-space the inner ring sits at small X_3, the outer at large X_3, so the plane $X_3 = c$ separates them perfectly. A curved boundary in $\mathbb{R}^2$ is a flat one in $\mathbb{R}^3$.

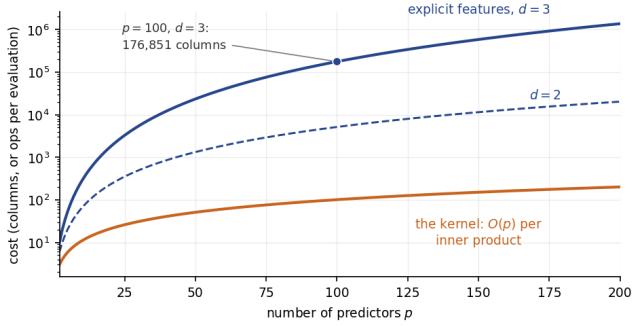
Why this does not scale

All monomials up to degree d in p predictors number $\binom{p+d}{d}$. With $p = 100$, $d = 3$ that is 176 851 columns; at $d = 4$, over four million. We cannot build, store or fit that design matrix.

Takeaway

The kernel trick delivers the enlarged space *without ever constructing it*.

What the enlargement costs, drawn



Takeaway

Monomials up to degree d in p variables: $\binom{p+d}{d}$ columns. At $p = 100$, $d = 3$ the explicit design matrix has 176 851 columns; the kernel computes the *same* inner product in $O(p)$ operations. That gap is the kernel trick.

The kernel trick

From inner products to kernels

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i \langle x, x_i \rangle \quad \longrightarrow \quad f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i), \qquad K(x, x') = \langle \phi(x), \phi(x') \rangle$$

How to read this

- Fitting *and* prediction touch the data only through pairwise inner products. Replace that one function and you have replaced the feature space.
- A **kernel** K is any similarity that equals an inner product in *some* enlarged space $\phi(\cdot)$; symmetric and positive semi-definite suffices, and we never need to know ϕ.
- $\alpha_i = 0$ off the support vectors, so the stored model is small. A support vector classifier with a non-linear kernel is a **support vector machine**.

Takeaway

Cost is set by n (an $n \times n$ kernel matrix), not by the dimension of the enlarged space — which may be infinite.

The three kernels you will actually use

Definitions

$$\underbrace{\langle x, x' \rangle}_{\text{linear}} \qquad \underbrace{\big(1 + \langle x, x' \rangle\big)^d}_{\text{polynomial, } d=1,2,3,\ldots} \qquad \underbrace{\exp\big(-\gamma \|x - x'\|^2\big)}_{\text{radial (RBF), } \gamma>0}$$

What each buys

- **Linear** ($d = 1$) reproduces the support vector classifier; the right default when p is large.
- **Polynomial** of degree d spans all monomials up to degree d; ill-behaved numerically for $d \geq 4$ and rarely worth it.
- **Radial** corresponds to an *infinite*-dimensional ϕ and is the workhorse: `kernel='rbf'`, parameter `gamma`.

Common mistake

`SVC(kernel='poly', degree=2)` does *not* match the formula above: sklearn uses $(\gamma\langle x, x' \rangle + r)^d$ with $r =$`coef0`$= 0$ by default, which kills the lower-order terms. Set `coef0=1` (and `gamma=1`) for ISLP's kernel.

The radial kernel: γ sets how local the boundary is

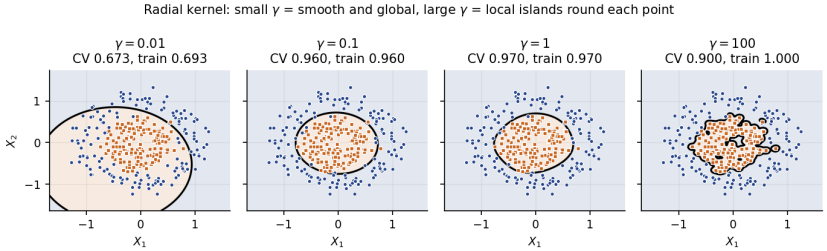
Reading the radial kernel

$$K(x, x') = \exp\big(-\gamma \|x - x'\|^2\big) \in (0, 1]$$

How to read this

- K equals 1 when $x = x'$ and decays towards 0 as the points separate; γ sets *how fast*.
- **Small** γ: distant points still count, every support vector votes everywhere, the boundary is smooth and nearly linear — high bias.
- **Large** γ: only near neighbours count, so each support vector carves out its own island — high variance. The limit is a nearest-neighbour rule.
- $\|x - x'\|^2$ is a *squared distance*, so γ carries units of $1/(\text{distance})^2$: it is meaningless without a fixed scale for the predictors.

What γ does to the boundary



Takeaway

At $C = 1$ on the concentric data: $\gamma = 0.01$ is too smooth to see the ring at all (CV 0.673); $\gamma = 0.1$ and 1 recover it (0.960, 0.970); $\gamma = 100$ fits the training set *perfectly* (1.000) by growing islands round individual points, and CV accuracy *drops* to 0.900. Textbook overfitting.

C and γ pull on the same rope

Two knobs, one flexibility

- γ controls the *shape*: how wiggly the boundary may be.
- sklearn's C controls how hard the fit *insists* that this shape reproduce the training labels.
- Large γ *and* large C is the fastest route to a memorised training set; small γ *and* small C is almost a linear rule.

Takeaway

They interact, so they must be tuned **jointly** on a grid, never one after the other. And because γ is tied to the *scale* of the features, a fitted pair (C, γ) is only valid alongside the scaler it was tuned with: freeze the two together and re-tune on a schedule.

Scaling is not optional

Heart data: what unscaled predictors do to a radial kernel

The 16 predictors have wildly different spreads — `Chol` has standard deviation 52.0, `Fbs` has 0.35. The median squared distance between two patients is $\|x - x'\|^2 \approx 4\,294$ raw against 31.1 standardised, so at $\gamma = 0.05$ a typical kernel entry is $e^{-0.05 \times 4294} = 5.8 \times 10^{-94}$ raw against 0.211 standardised. The raw kernel matrix is numerically the identity.

Takeaway

On the Heart test set the identical model (`rbf`, $C = 1$, $\gamma = 0.05$) scores 0.556 unscaled and 0.822 scaled. Unscaled it predicts "No" for 86 of 90 test patients — it has collapsed to the majority class.

Common mistake

Standardising *before* the train/test split, or outside the cross-validation loop. Put `StandardScaler` inside a `Pipeline` so every fold rescales with its own training rows only.

Exercise 9.4 — The polynomial kernel is an inner product [Math]

Exercise

Let $p = 2$ and $K(x, x') = \big(1 + \langle x, x' \rangle\big)^2$ with $x = (x_1, x_2)$, $x' = (x_1', x_2')$.

1. Expand $K(x, x')$ fully in the four coordinates.
2. Exhibit an explicit map $\phi : \mathbb{R}^2 \to \mathbb{R}^m$ with $K(x, x') = \langle \phi(x), \phi(x') \rangle$. What is m?
3. How many multiplications does each route need for one kernel entry?
4. The enlarged space has $\binom{p+d}{d}$ dimensions. Evaluate this for $p = 100$, $d = 3$, and state the cost of the kernel route instead.

Solution — Exercise 9.4 (1/2)

Solution

Part 1. With $s = x_1 x_1' + x_2 x_2'$,

$$K = (1+s)^2 = 1 + 2s + s^2 = 1 + 2x_1 x_1' + 2x_2 x_2' + x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_2 x_1' x_2'.$$

Part 2. Read the six terms as one inner product, splitting each factor 2 as $\sqrt{2} \cdot \sqrt{2}$:

$$\phi(x) = \left(1,\ \sqrt{2}\,x_1,\ \sqrt{2}\,x_2,\ x_1^2,\ x_2^2,\ \sqrt{2}\,x_1 x_2\right) \in \mathbb{R}^6, \qquad m = 6 = \binom{4}{2}. \checkmark$$

Solution — Exercise 9.4 (2/2)

Solution

Part 3. Via the kernel: two products for s, then one squaring — **three** multiplications. Via ϕ: **six** multiplications for the inner product, plus building both 6-vectors and storing 6 numbers per observation instead of 2.

Part 4. $\binom{103}{3} = (103 \cdot 102 \cdot 101)/6 = 176\,851$ columns in the enlarged design matrix, versus $p + 1 = 101$ multiplications per pair for the kernel and no extra storage — the *same fit* at a cost independent of the enlarged dimension.

Common mistake

Dropping the $\sqrt{2}$ factors and writing $\phi(x) = (1, x_1, x_2, x_1^2, x_2^2, x_1 x_2)$. That map reproduces the cross terms with the wrong weights, so it is a *different* kernel.

Exercise 9.5 — Kernels on the concentric data [Python]

Exercise

Generate `X, y = make_circles(n_samples=300, factor=0.4, noise=0.15, random_state=0)`.

1. In a `Pipeline` with `StandardScaler`, tune a **linear** SVM over `C = np.logspace(-2, 2, 9)` by 5-fold CV. Report the best C and CV accuracy.
2. Tune a **radial** SVM over `C = np.logspace(-2, 2, 5)` and `gamma = np.logspace(-3, 1, 5)`.
3. Fit a **polynomial** kernel of degree 2 with `coef0=1, C=1`. Does it match the radial kernel?
4. Explain the gap between (1) and (2) in one sentence.

Run this live

The worked solution sits in `advanced_05_svm_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Exercise 9.5 — Kernels on the concentric data [Python]*.

Solution — Exercise 9.5 (1/2)

Solution — code

```
import numpy as np
from sklearn.datasets import make_circles # rings: no line works
from sklearn.model_selection import GridSearchCV, cross_val_score # tune / score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler # kernels need one scale
from sklearn.svm import SVC

X, y = make_circles(n_samples=300, factor=0.4, noise=0.15, random_state=0)
pipe = lambda **kw: make_pipeline(StandardScaler(), SVC(**kw)) # scale, then fit

lin = GridSearchCV(pipe(kernel='linear'), # (1) linear
                   {'svc__C': np.logspace(-2, 2, 9)}, cv=5).fit(X, y) # 9 C values
rbf = GridSearchCV(pipe(kernel='rbf'), # (2) radial
                   {'svc__C': np.logspace(-2, 2, 5),
                    'svc__gamma': np.logspace(-3, 1, 5)}, cv=5).fit(X, y) # 25 cells
pol = cross_val_score(pipe(kernel='poly', degree=2, coef0=1, C=1), X, y, cv=5)

print('linear', lin.best_params_, round(lin.best_score_, 3)) # 0.583: coin flip
print('radial', rbf.best_params_, round(rbf.best_score_, 3)) # 0.980: shape fixed
print('poly d=2 CV', round(pol.mean(), 3)) # 0.963: a circle is degree 2
```

Reading it: one `Pipeline` per kernel, one 5-fold CV — each line prints the winning cell and its CV accuracy.

Solution — Exercise 9.5 (2/2)

Solution — output, and how to read it

Printout.

```
linear {'svc__C': 0.01} 0.583   radial {'svc__C': 1.0, 'svc__gamma':  10.0} 0.98   poly d=2  CV
                                                                          0.963
```

- **(1)** The best linear SVM reaches 0.583; with a 50/50 class split that is barely better than a coin flip. Note CV picked the *smallest* C: when no line can work, a wide margin at least keeps the fit stable.
- **(2)** The radial kernel reaches 0.980 at $C = 1$, $\gamma = 10$ — a 0.40 jump in accuracy from one change of kernel.
- **(3)** Degree 2 scores 0.963, and that is no accident: the true boundary is a circle, exactly a degree-2 surface. The radial kernel wins slightly because it can also absorb noise in the ring's radius.
- **(4)** The classes are *concentric*, so no half-space contains one class: the failure is the boundary's *shape*, and only a change of kernel — not of C — can fix it.

Exercise 9.6 — Scaling the Heart predictors [Python]

Exercise

Load `Heart.csv` with `index_col=0`, drop rows with missing values, code `AHD` as 0/1, one-hot encode `ChestPain` and `Thal` with `drop_first=True`, and split with `test_size=0.3, random_state=0`.

1. Fit `SVC(kernel='rbf', C=1, gamma=0.05)` on the *raw* predictors; report the test accuracy and the class counts it predicts.
2. Fit the same model inside `make_pipeline(StandardScaler(), ...)` and report the test accuracy again.
3. Compute the median squared distance between pairs of patients, raw and standardised, and use it to explain the difference.

Run this live

Worked in `advanced_05_svm_lab.ipynb` under *Lecture exercises — worked Python solutions*: *Exercise 9.6 — Scaling the Heart predictors [Python]*.

Solution — Exercise 9.6 (1/2)

Solution — code

```
import numpy as np, pandas as pd
from scipy.spatial.distance import pdist # pairwise squared distances
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler # the whole point of (2)
from sklearn.svm import SVC

H = load('Heart', index_col=0).dropna().reset_index(drop=True) # 297 rows
y = (H['AHD'] == 'Yes').astype(int).values # 1 = heart disease
X = pd.get_dummies(H.drop(columns='AHD'), drop_first=True).astype(float).values
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

raw = SVC(kernel='rbf', C=1, gamma=0.05).fit(Xtr, ytr) # (1) unscaled
sc = make_pipeline(StandardScaler(), # (2) scaled
                   SVC(kernel='rbf', C=1, gamma=0.05)).fit(Xtr, ytr)
print('unscaled', round(raw.score(Xte, yte), 3), np.bincount(raw.predict(Xte)))
print('scaled ', round(sc.score(Xte, yte), 3)) # 0.822: +27 points
print('median d^2: raw %.0f scaled %.1f' # (3) why
     % (np.median(pdist(X, 'sqeuclidean')), # raw: 4294
        np.median(pdist(StandardScaler().fit_transform(X), 'sqeuclidean'))))
```

Reading it: (1) and (2) differ only by the scaler; (3) prints the distance the radial kernel sees.

Solution — Exercise 9.6 (2/2)

Solution — output, and how to read it

Printout.

```
unscaled 0.556 [86  4]   scaled 0.822   median d^2:  raw 4294  scaled 31.1
```

- **(1)** 0.556 test accuracy, with only 4 of 90 patients predicted "Yes". The test set is 46.7% "Yes", so this is essentially the majority-class rule.
- **(2)** The identical model, with scaling, reaches 0.822 — a 27-point gain from three words of code.
- **(3)** $\exp(-0.05 \times 4294) = 5.8 \times 10^{-94}$: raw, every off-diagonal kernel entry underflows to zero, so $f(x) \approx \beta_0$. Standardised, $\exp(-0.05 \times 31.1) = 0.211$ — a usable similarity.

Common mistake

Calling `StandardScaler().fit_transform(X)` on the full data before splitting. The means and variances then leak across the split; use a `Pipeline`.

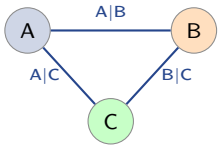
Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

More than two classes: one-vs-one and one-vs-all

A hyperplane has two sides, so there is no natural K-class version. Both standard fixes reduce the problem to a family of binary ones.

One-vs-one (OVO), $K = 3$



$\binom{K}{2} = K(K-1)/2$ fits, majority vote

One-vs-all (OVA), $K = 3$

A vs {B,C}

B vs {A,C}

C vs {A,B}

K fits, largest $f_k(x)$ wins

Takeaway

`SVC` uses **OVO**; `LinearSVC` uses OVA. Neither is a principled multi-class objective — both are *voting schemes* bolted onto a binary method, and they can disagree. OVO's votes can tie; OVA compares scores from models fitted on different, badly unbalanced problems.

Exercise 9.7 — Counting multi-class classifiers [Concept]

Exercise

1. How many binary classifiers do OVO and OVA need for $K = 4$? For $K = 10$?
2. With $K = 10$ balanced classes and $n = 10\,000$, how many observations does one OVO fit use, and how many does one OVA fit use?
3. Kernel SVM training cost grows roughly like the square of the training-set size. Which scheme does less total work at $K = 10$?
4. Name one respect in which OVA is nonetheless the more honest construction.

Solution — Exercise 9.7

Solution

Part 1. OVO needs $\binom{K}{2} = K(K-1)/2$: $\binom{4}{2} = 6$ and $\binom{10}{2} = 45$. OVA needs K: 4 and 10.
Part 2. Balanced classes give $n/K = 1\,000$ per class. An OVO fit sees two classes, so 2 000 observations; an OVA fit sees all 10 000.
Part 3. With cost $\propto m^2$ in the sub-sample size m:

$$\text{OVO: } 45 \times 2\,000^2 = 1.8 \times 10^8, \qquad \text{OVA: } 10 \times 10\,000^2 = 1.0 \times 10^9.$$

OVO does about 5.6 times *less* work despite fitting 4.5 times as many models — which is why `SVC` defaults to it.
Part 4. OVA produces exactly one score $f_k(x)$ per class, so the prediction is a single arg max with no ties and no vote-counting, and each fit answers a question we care about ("is it class k?"). Its weakness mirrors OVO's: each fit is badly unbalanced (1 against 9), and the K scores are not on a comparable scale.

Outline

1. Why This Chapter
2. Hyperplanes
3. Maximal Margin
4. Soft Margin
5. Kernels
6. Many Classes
7. Tuning and Comparison
8. Python Lab
9. Summary

Tuning C and γ by cross-validation (bridge to Chapter 5)

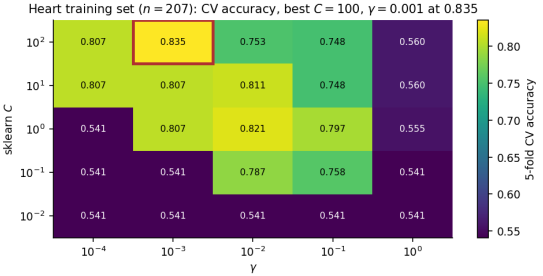
The recipe

1. Put `StandardScaler` and `SVC` in a `Pipeline`, so scaling is refitted inside every fold.
2. Lay out a *logarithmic* grid, e.g. $C \in \{10^{-2}, \ldots, 10^{2}\}$, $\gamma \in \{10^{-4}, \ldots, 10^{0}\}$.
3. Score every cell by k-fold CV on the **training** data only.
4. Refit the winning cell on all the training data; report on the untouched test set.

How to read this

- Grids must be logarithmic: C and γ act multiplicatively, so $\{1, 2, 3\}$ explores almost nothing.
- If the winner sits on the edge of the grid, extend the grid — the optimum is outside it.
- Nested CV if you also want an honest error estimate for the *tuned* model; one held-out test set is the cheap version.

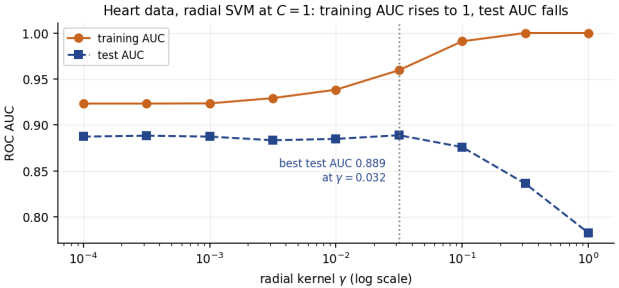
The (C, γ) surface on the Heart data



Takeaway

The best cell is $C = 100$, $\gamma = 10^{-3}$ at CV accuracy 0.835. The dark cells at 0.541 are exactly the training-set majority-class rate: there the model has degenerated to "always predict No". Note the diagonal ridge — large γ needs small C and vice versa, which is why they must be tuned together.

Flexibility is not free: γ and the test set



Takeaway

At $C = 1$, raising γ drives training AUC from 0.923 to a perfect 1.000 while test AUC *falls* from 0.887 to 0.782. The best test AUC, 0.889, comes at $\gamma \approx 0.03$ — a boundary barely more flexible than a straight line.

SVM vs. logistic regression: the honest comparison is the loss

Both minimise "loss + ridge penalty"

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} L\big(y_i f(x_i)\big) + \lambda\|\beta\|^2, \quad L(t) = \max\{0,\, 1-t\} \text{ (hinge, SVM)} \quad \text{or} \quad \log\big(1+e^{-t}\big) \text{ (logistic)}$$

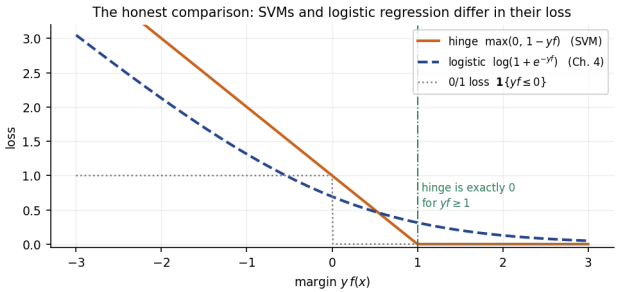
How to read this

- $t = y_i f(x_i)$ is the functional margin; $\lambda = 1/(2C)$ ties the penalty back to sklearn's C.
- The **hinge** loss is *exactly zero* for $t \geq 1$, so points comfortably right contribute nothing — that is the sparsity, and the support vectors, in one line.
- The **logistic** loss is never zero, so *every* observation nudges the fit, a little, forever.

Takeaway

Same linear model, same ridge penalty, different loss. That is the whole difference — not "geometry versus probability".

Hinge and logistic loss, drawn



Takeaway

Both are convex upper bounds on the 0/1 loss and nearly coincide near $t = 0$ — which is why the two methods usually agree. They differ at the ends: hinge switches off at $t = 1$ (sparsity), logistic keeps a tail (probabilities, and sensitivity to far-away points).

SVMs give you no probabilities

What you get, and what you do not

- `decision_function(X)` returns $f(x)$: a real-valued, monotone *score*. That is all an ROC curve or an AUC needs, so Chapter 4's ranking machinery still applies.
- `predict_proba` does not exist unless you pass `SVC(probability=True)`, which fits a **separate** logistic calibration (Platt scaling) by internal 5-fold CV.

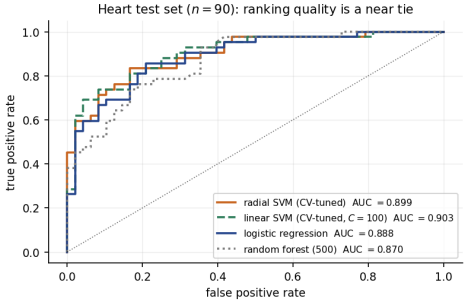
Common mistake

Using `SVC(probability=True).predict_proba(...)[:,1]` for an ROC curve. On the Heart data it happens to give the identical AUC (0.899; Spearman correlation 1.00 with f), but it is a different estimator, seeded, and several times slower to fit. **For ROC and AUC, always use `decision_function`.**

Takeaway

Need calibrated probabilities for an expected-cost decision? Calibrate deliberately, or use logistic regression, which gives them for free.

ROC on the Heart test set: SVM, logistic, forest



Takeaway

Four methods, 90 test patients, AUCs within 0.033: linear SVM 0.903, radial SVM 0.899, logistic regression 0.888, random forest 0.870. The *simplest* kernel wins — the radial one buys nothing here.

SVM vs. trees and forests (bridge to Chapter 8)

	SVM (radial)	**Logistic regression**	**Random forest**
Boundary shape	smooth, curved	linear in X	axis-aligned steps
Probabilities	none by default	yes	yes, from vote shares
Needs scaling	**yes, essential**	helpful for penalties	no
Categoricals	must be encoded	must be encoded	handled natively
$p \gg n$	very strong	needs a penalty	workable
Missing values	must be imputed	must be imputed	tolerant
Interpretation	weak	coefficients, odds ratios	importances
Tuning burden	C, γ, kernel	little	moderate
Heart test AUC	0.899	0.888	0.870

Takeaway

On 297 patients with 16 mixed predictors all three land inside 0.03 AUC of each other. Choose on the *secondary* criteria — probabilities, interpretability, missing data, tuning cost — because accuracy will not decide it for you.

When to reach for an SVM

Decision guidance

- **Reach for it** when p is large relative to n, when the classes are close to separable, when a bespoke similarity (string, graph, sequence kernel) is natural, or when you need a hard, reproducible boundary.
- **Reach elsewhere** when you need probabilities, when n is in the hundreds of thousands ($n \times n$ kernel matrices do not fit), when the data are mostly categorical with missing values, or when a stakeholder must read coefficients.

Why the SVM never became the default

Credit scorecards must give a monotone, explainable probability of default and survive a regulator asking "why was this applicant declined?"; penalised logistic regression and boosted trees with SHAP answer that, a radial-kernel SVM does not. The method stayed where explanation matters less and p/n is brutal — assays, spectra, text, novelty detection.

Extended Exercise 9.2 — Tune an SVM on Heart and race it [Python]

Extended exercise (15 min)

Using the Heart split of Exercise 9.6 (207 train, 90 test):

1. In a scaled `Pipeline`, tune a radial SVM by 5-fold CV over $C \in \{10^{-2}, \ldots, 10^{2}\}$ and $\gamma \in \{10^{-4}, \ldots, 10^{0}\}$ (five values each). Report the best cell, its CV and test accuracy, and its number of support vectors.
2. Do the same for a *linear* kernel over the same C grid.
3. Fit a scaled logistic regression and a 500-tree random forest.
4. Report the test ROC AUC of all four, using `decision_function` for the SVMs and `predict_proba` for the others. Which wins, and by how much?
5. What do you conclude about the value of the radial kernel here?

Run this live

Worked in `advanced_05_svm_lab.ipynb` under *Lecture exercises — worked Python solutions*: *Extended Exercise 9.2*.

Solution — Extended Exercise 9.2 (1/2)

Solution — code

```
Cs = np.array([0.01, 0.1, 1.0, 10.0, 100.0]) # log grid, 5 values
Gs = np.array([1e-4, 1e-3, 1e-2, 1e-1, 1.0]) # log grid, 5 values
pipe = lambda **kw: make_pipeline(StandardScaler(), SVC(**kw)) # scale in-fold
rbf = GridSearchCV(pipe(kernel='rbf'),
                   {'svc__C': Cs, 'svc__gamma': Gs}, cv=5).fit(Xtr, ytr) # 25 cells
lin = GridSearchCV(pipe(kernel='linear'), {'svc__C': Cs}, cv=5).fit(Xtr, ytr)
for nm, gs in [('radial', rbf), ('linear', lin)]: # report both fits
    print(nm, gs.best_params_, 'CV', round(gs.best_score_, 3),
          'test', round(gs.score(Xte, yte), 3),
          'nSV', gs.best_estimator_[-1].n_support_.sum()) # 80 vs 72

lg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)).fit(Xtr, ytr)
rf = RandomForestClassifier(n_estimators=500, random_state=0).fit(Xtr, ytr)
scores = {'radial SVM': rbf.best_estimator_.decision_function(Xte), # NOT proba
          'linear SVM': lin.best_estimator_.decision_function(Xte),
          'logistic': lg.predict_proba(Xte)[:, 1], # a real probability
          'forest': rf.predict_proba(Xte)[:, 1]} # vote share, 500 trees
for nm, sc in scores.items(): # roc_auc_score
    print('%-11s AUC=%.3f' % (nm, roc_auc_score(yte, sc))) # order is all it needs
```

Reading it: both SVMs are tuned on the training split; all four are then compared by AUC on the test split.

Solution — Extended Exercise 9.2 (2/2)

Solution — output, and what it means

Printout.

```
radial {'svc__C': 100.0, 'svc__gamma':  0.001} CV 0.835 test 0.789 nSV 80
linear {'svc__C': 100.0} CV 0.84 test 0.822 nSV 72
radial SVM  AUC=0.899   linear SVM  AUC=0.903   logistic   AUC=0.888   forest    AUC=0.870
```

- **(1)–(2)** The tuned radial kernel picks $\gamma = 10^{-3}$, so small that the kernel is *nearly linear*: CV has itself told us the extra flexibility is unwanted.
- **(4)** All four AUCs lie in [0.870, 0.903] — a spread of 0.033 on 90 test patients, inside sampling noise at this size.
- **(5)** The radial kernel buys *nothing*: the linear SVM is at least as good on CV accuracy (0.840 vs. 0.835) and test AUC (0.903 vs. 0.899), with fewer support vectors and one hyperparameter instead of two. Prefer it.

Common mistake

Reading the winner of a 0.033 AUC gap on $n = 90$ as a real finding. Quote the spread, or bootstrap a confidence interval, before declaring a champion.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

Python lab — run it live in the notebook

Companion notebook: `advanced_05_svm_lab.ipynb`

The code in this section is meant to be **demonstrated live**. Open the companion Jupyter notebook and run it cell by cell:

- §1, *Synthetic two-class data* — the concentric rings from `make_circles`;
- §2, *Linear SVM* — the margin, the support vectors and sklearn's C;
- §3, *Radial-kernel SVM* — γ, and the decision boundaries side by side;
- §4, *Real data: Heart* — encoding, scaling, and the unscaled disaster;
- §5, *Tuning C and γ by cross-validation* — the grid, the heatmap and the ROC race.

Data loads via the `ISLP` package or the bundled CSV files; §6, *Exercises* ends the notebook with hands-on tasks.

A support vector classifier from start to finish

```
import numpy as np
from sklearn.pipeline import make_pipeline # scaler + model as one
from sklearn.preprocessing import StandardScaler # SVMs NEED this
from sklearn.svm import SVC # large C = narrow street

rng = np.random.default_rng(2024) # seeded: reproducible
X = rng.normal(size=(20, 2)) # 20 points in the plane
y = np.repeat([-1, 1], 10) # +-1 labels, 10 each
X[y == 1] += 2.5 # now separable by a line

hard = SVC(kernel='linear', C=1e6).fit(X, y) # huge C ~hard margin
print('n support vectors:', hard.n_support_.sum()) # 3 of 20 shape the fit
print('margin 1/||beta|| :', round(1 / np.linalg.norm(hard.coef_), 3)) # 0.369
```

Takeaway

`C=1e6` is how you ask `SVC` for a (near) hard margin — remember, *large* sklearn C means *narrow* margin.

A radial-kernel SVM, tuned, and its ROC score

```
from sklearn.datasets import make_circles # rings, not separable
from sklearn.metrics import roc_auc_score # ranks, not labels
from sklearn.model_selection import GridSearchCV # refits on the best cell

X, y = make_circles(n_samples=300, factor=0.4, noise=0.15, random_state=0)
grid = {'svc__C': np.logspace(-2, 2, 5), # log grid, 5 x 5 cells
        'svc__gamma': np.logspace(-3, 1, 5)} # gamma: locality
gs = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel='rbf')),
                  grid, cv=5).fit(X, y) # scaling inside each fold
print('best:', gs.best_params_, 'CV:', round(gs.best_score_, 3)) # C=1, g=10

s = gs.best_estimator_.decision_function(X) # no predict_proba needed
print('in-sample AUC:', round(roc_auc_score(y, s), 3)) # a ranking is enough
```

Common mistake

Scaling once outside `GridSearchCV`: every fold's validation rows have then already influenced the mean and variance used to transform them — a small but real leak that flatters the CV score.

Outline

1 Why This Chapter
2 Hyperplanes
3 Maximal Margin
4 Soft Margin
5 Kernels
6 Many Classes
7 Tuning and Comparison
8 Python Lab
9 Summary

Module A5 in one slide

What we accomplished

Classification as geometry: the widest street between two classes, made robust by a budget and made non-linear by a kernel.

- Separating hyperplanes, the score $f(x)$, the functional margin $y_i f(x_i)$.
- The maximal margin classifier, $M = 1/\|\beta\|$, and its two fatal flaws.
- The support vector classifier: slacks ξ_i, a budget on $\sum_i \xi_i$, and sklearn's inverted C.
- Support vector machines: the kernel trick, degree d, and γ.
- One-vs-one and one-vs-all for $K > 2$ classes.
- Tuning (C, γ) by cross-validation; hinge vs. logistic loss; ROC from `decision_function`.

Key formulas at a glance

Hyperplane $\{x : \beta_0 + \beta^\top x = 0\}$; classifier $\hat{y} = \operatorname{sign} f(x)$, $f(x) = \beta_0 + \beta^\top x$.

Signed distance $f(x_0)/\|\beta\|$; correct classification $\iff y_i f(x_i) > 0$.

Maximal margin $\max M$ s.t. $\|\beta\| = 1$, $y_i f(x_i) \geq M \ \forall i$.

Canonical form $\min \frac{1}{2}\|\beta\|^2$ s.t. $y_i f(x_i) \geq 1$, giving $M = 1/\|\beta\|$.

Soft margin (ISLP) $\max M$ s.t. $\|\beta\| = 1$, $y_i f(x_i) \geq M(1 - \xi_i)$, $\xi_i \geq 0$, $\sum_{i=1}^n \xi_i \leq C_{\text{ISLP}}$.

Soft margin (sklearn) $\min \frac{1}{2}\|\beta\|^2 + C \sum_i \xi_i$ s.t. $y_i f(x_i) \geq 1 - \xi_i$, $\xi_i \geq 0$; slack $\xi_i = \max\{0, 1 - y_i f(x_i)\}$.

Penalised form $\min \sum_i \max\{0, 1 - y_i f(x_i)\} + \lambda\|\beta\|^2$, $\lambda = 1/(2C)$.

Dual / kernel rule $f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$, $\alpha_i \neq 0$ only on $\mathcal{S}$.

Kernels linear $\langle x, x' \rangle$; polynomial $(1 + \langle x, x' \rangle)^d$; radial $\exp(-\gamma\|x - x'\|^2)$.

Enlarged dimension $\binom{p+d}{d}$ monomials up to degree d ($\binom{4}{2} = 6$ for $p = d = 2$; $\binom{103}{3} = 176\,851$ for $p = 100$, $d = 3$).

Losses hinge $\max\{0, 1 - t\}$ vs. logistic $\log(1 + e^{-t})$, $t = y_i f(x_i)$.

Multi-class OVO: $\binom{K}{2} = K(K-1)/2$ fits, majority vote; OVA: K fits, $\hat{y} = \arg\max_k f_k(x)$.

Vocabulary checklist

Term	Meaning
Hyperplane	Flat $(p-1)$-dimensional boundary $\beta_0 + \beta^\top x = 0$
Functional margin	$y_i f(x_i)$; positive iff correctly classified
Margin M	Distance from boundary to nearest point; $1/\|\beta\|$ canonically
Support vector	Observation with $y_i f(x_i) \leq 1$; the only ones that matter
Slack ξ_i	$\max\{0, 1 - y_i f(x_i)\}$; > 1 means misclassified
Budget C_{ISLP}	Cap on $\sum_i \xi_i$; larger = wider margin
sklearn C	Cost per unit slack; larger = *narrower* margin
Support vector classifier	Soft margin with a linear kernel
Support vector machine	Soft margin with a non-linear kernel
Kernel $K(x, x')$	Inner product in an implicit enlarged space ϕ
γ	Radial-kernel locality; large γ = wiggly boundary
Hinge loss	$\max\{0, 1 - t\}$; zero once a point is safely right
`decision_function`	The score $f(x)$; what ROC curves need
Platt scaling	Post-hoc logistic calibration behind `probability=True`
OVO / OVA	One-vs-one / one-vs-all multi-class wrappers

Decision rules of thumb

- **Always** put a `StandardScaler` in front of an SVM, inside a `Pipeline`.
- Start with a **linear** kernel; move to radial only if CV says the extra flexibility pays.
- Tune C and γ *jointly* on a logarithmic grid; extend the grid if the winner sits on its edge.
- Many support vectors $\Rightarrow$ wide margin $\Rightarrow$ high bias, low variance. Few $\Rightarrow$ the reverse.
- Report ROC AUC from `decision_function`; reach for `probability=True` only when you genuinely need probabilities.
- n in the hundreds of thousands? Use `LinearSVC` or `SGDClassifier(loss='hinge')` — an $n \times n$ kernel matrix will not fit.
- Must explain a decision to a regulator or a clinician? Prefer penalised logistic regression or a boosted tree.

Common pitfalls

Don't

1. Confuse ISLP's budget C_{ISLP} with sklearn's cost C — they run in *opposite* directions.
2. Fit an SVM on unscaled predictors, or scale outside the CV loop.
3. Say "the budget limits how many points are misclassified". It limits $\sum_i \xi_i$.
4. Treat many support vectors as a sign of overfitting.
5. Read $|f(x)|$ as a probability, or as a distance without dividing by $\|\beta\|$.
6. Draw an ROC curve from `SVC(probability=True)` instead of `decision_function`.
7. Tune γ and C one at a time, or on a linear grid.
8. Use `kernel='poly'` without `coef0=1` and expect ISLP's $(1 + \langle x, x' \rangle)^d$.
9. Declare a winner from a 0.03 AUC gap on 90 test observations.

Self-check questions

Quick self-assessment

1. Why does the maximal margin problem impose $\|\beta\| = 1$, and what replaces that constraint in the form software solves?
2. An observation has $\xi_i = 0.6$. Is it misclassified? Is it a support vector?
3. You double sklearn's C. What happens to the margin, to $\|\beta\|$, and to $|\mathcal{S}|$?
4. Why can a radial kernel work when its enlarged feature space is infinite-dimensional?
5. Which loss — hinge or logistic — is zero for some observations, and why does that produce sparsity?

Answers

(1) To make $y_i f(x_i)$ a true distance; the canonical scaling $y_i f(x_i) \geq 1$ replaces it, and then $M = 1/\|\beta\|$. (2) Not misclassified (that needs $\xi_i > 1$), but yes, a support vector. (3) Margin narrows, $\|\beta\|$ grows, $|\mathcal{S}|$ falls. (4) Because fitting and prediction touch the data only through the $n \times n$ kernel matrix, never through ϕ. (5) Hinge: it is exactly 0 once $y_i f(x_i) \geq 1$, so those points drop out of the solution and get $\alpha_i = 0$.

Eight things to remember

Module A5 in eight lines

1. An SVM is a *boundary*, not a probability model.
2. Maximising the margin $=$ minimising $\|\beta\|$; $M = 1/\|\beta\|$.
3. Only the support vectors ($y_i f(x_i) \leq 1$) shape the fit.
4. The soft margin budgets the **sum** of the slacks.
5. sklearn's C is *inverse* regularisation: big C, narrow street.
6. Kernels buy a huge feature space at the cost of an $n \times n$ matrix.
7. γ sets locality; scale your predictors or γ means nothing.
8. SVM and logistic regression differ only in their loss — and usually agree.

What's next

Module A9 (deep learning, ISLP Chapter 10) stops *choosing* the enlarged features and *learns* them instead: a neural network is a stack of adaptive basis functions with a linear rule on top.

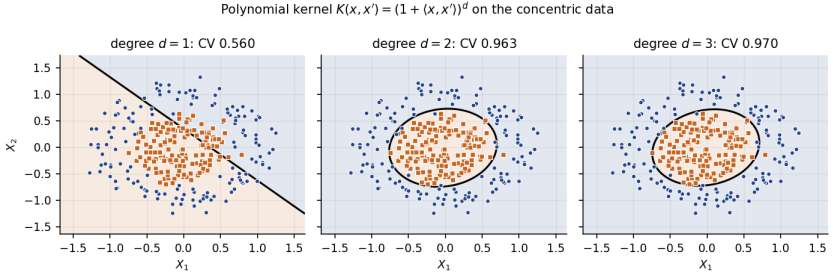
Appendix — what is in here, and why

Nothing in the appendix is needed to follow the chapter: each slide is a formal derivation, a longer worked exercise, or a side topic. Read it when you want the full story, or when a later chapter sends you back here.

Contents of the appendix

- **The polynomial kernel: what degree d does** — the degree sweep drawn; Exercise 9.5 reports the same finding numerically (degree 2 scores 0.963 on a circular boundary).
- **A hyperplane in the plane** — the picture behind Exercise 9.1, moved out because the main deck needs the concept, not the drawing.
- **Distance to a hyperplane** — the two-line derivation behind $|f(x_0)|/\|\beta\|$, which the main deck simply asserts.
- **Why $M = 1/\|\beta\|$** — the algebra that turns "maximise the margin" into "minimise $\|\beta\|^2$".
- **The dual problem** — where the α_i come from and why they vanish off the support vectors. This is the machinery behind the kernel trick; examinable only at the level of the statement.
- **Support vector regression** — the same idea for a quantitative response, with an ϵ-insensitive tube. A side topic, outside ISLP Chapter 9's core.
- **Extended Exercise 9.3** — a design exercise on choosing between SVM, logistic regression and a forest. Run it if the session has time; it needs no new theory.

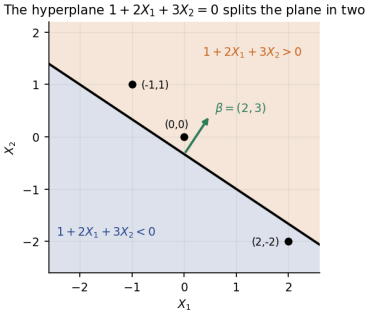
The polynomial kernel: what degree d does



Takeaway

$d = 1$ is just a line and scores 0.560; $d = 2$ can bend into an ellipse and jumps to 0.963; $d = 3$ reaches 0.970. Almost all the gain comes from the *first* step past linearity, because the true boundary here is a circle — a degree-2 shape.

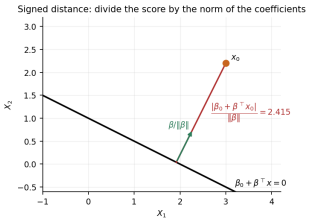
A hyperplane in the plane



Takeaway

Points with $1 + 2X_1 + 3X_2 > 0$ sit in the orange half-space, points with < 0 in the blue one. The green arrow is $\beta = (2, 3)$: perpendicular to the line, pointing into the positive half-space. The three marked points are the ones classified in Exercise 9.1.

Distance from a point to a hyperplane



The derivation

Let x_p be any point on the plane, so $\beta_0 + \beta^\top x_p = 0$, and let $u = \beta/\|\beta\|$ be the unit normal. The distance from x_0 to the plane is the length of the projection of $x_0 - x_p$ onto u:

$$u^\top (x_0 - x_p) = \frac{\beta^\top x_0 - \beta^\top x_p}{\|\beta\|} = \frac{\beta^\top x_0 + \beta_0}{\|\beta\|} = \frac{f(x_0)}{\|\beta\|},$$

using $\beta^\top x_p = -\beta_0$. The absolute value is the distance; the sign says which side.

Why $M = 1/\|\beta\|$

From "maximise M" to "minimise $\|\beta\|^2$"

- Suppose $y_i f(x_i) \geq M$ for all i with $\|\beta\| = 1$. By the previous slide, M is then the smallest distance from the plane to a data point.
- Now drop $\|\beta\| = 1$ and rescale (β_0, β) by $1/M$. The constraints become $y_i f(x_i) \geq 1$, and the new coefficient vector has norm $\|\beta\|_{\text{new}} = 1/M$.
- So maximising M over unit-norm β is exactly minimising $\|\beta\|$ subject to $y_i f(x_i) \geq 1$, with $M = 1/\|\beta\|$ recovered at the end.
- Squaring and halving — minimise $\frac{1}{2}\|\beta\|^2$ — leaves the minimiser unchanged and makes the objective a smooth quadratic, which is what QP solvers want.

Takeaway

This is why the chapter can be read as ridge regression with a different loss: the SVM's primary objective *is* a penalty on $\|\beta\|^2$.

The dual problem, and where the α_i come from

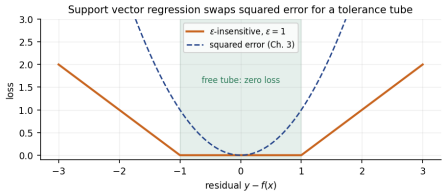
Wolfe dual of the soft-margin problem

$$\max_{a} \sum_{i=1}^{n} a_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{i'=1}^{n} a_i a_{i'} y_i y_{i'} K(x_i, x_{i'}) \quad \text{s.t.} \quad 0 \le a_i \le C, \quad \sum_{i=1}^{n} a_i y_i = 0$$

How to read this

- Write $\alpha_i = a_i y_i$; then $f(x) = \beta_0 + \sum_i \alpha_i K(x, x_i)$, the rule from the main deck. The data appear *only* inside $K(x_i, x_{i'})$ — that is the whole content of the kernel trick.
- Complementary slackness gives three cases: $a_i = 0$ (outside the street, $y_i f(x_i) > 1$), $0 < a_i < C$ (exactly on the kerb), $a_i = C$ (inside the street or misclassified).
- So $a_i = 0$ off the support vectors: sparsity is a *consequence* of the hinge loss's flat region, not an assumption — and the geometric, mechanical and algebraic descriptions of a support vector are one reading of these same conditions.

Support vector regression (side topic)



The ϵ-insensitive loss

Replace the hinge loss by $L_\epsilon(r) = \max\{0, |r| - \epsilon\}$ on the residual $r = y - f(x)$ and keep the $\|\beta\|^2$ penalty. Residuals inside the tube of width 2ϵ cost *nothing*, so again only points on or outside the tube become support vectors. In scikit-learn this is `SVR`, with `epsilon` plus the same `C`, `gamma`.

Takeaway

Same three ingredients as the classifier — a flat-bottomed loss, a ridge penalty, a kernel — and the same sparsity. Robust to outliers in a way squared error is not, but it too returns no uncertainty.

Extended Exercise 9.3 — Choosing a classifier [Integrative]

Extended exercise (15 min)

For each brief, choose between a radial-kernel SVM, penalised logistic regression and a random forest. Justify the choice with this chapter's criteria (scaling, p versus n, probabilities, missing data, interpretability, tuning cost), and name the one thing that would change your mind.

1. **A** — A proteomics assay: $n = 90$ patients, $p = 3\,000$ peak intensities on a comparable continuous scale, no missing values. The output is a two-class research finding.
2. **B** — A consumer credit decision: $n = 400\,000$ applications, $p = 45$ mixed numeric and categorical predictors with missing income, and a regulator who asks why an applicant was declined. An expected-loss calculation needs Pr(default).
3. **C** — A factory novelty detector: $n = 12\,000$ vibration traces, $p = 60$ engineered spectral features, almost all traces normal, and the alarm threshold set later by operations.

Solution — Extended Exercise 9.3 (1/2)

Solution — briefs A and B

A — an SVM, starting linear. $p/n = 33$, so the data are certainly separable: unpenalised logistic regression will not converge, and a forest has little to split on with 90 rows. The margin is well defined even here, the features are already on one scale (standardise anyway), and the kernel matrix is 90×90 — free. Start *linear*: with $p \gg n$ a linear kernel is usually as good as radial and has one hyperparameter. *What would change my mind:* if the finding needed a per-patient probability for a clinical decision, switch to penalised logistic regression.

B — penalised logistic regression (or a boosted tree with SHAP). Three deal-breakers for the SVM: $n = 400\,000$ makes an $n \times n$ kernel matrix impossible (1.6×10^{11} entries); the decision needs a calibrated Pr(default), which an SVM does not give; and adverse action notices need per-applicant reason codes, which coefficients supply and support vectors do not. Missing income and categorical predictors also favour the GLM/tree side. *What would change my mind:* nothing about the model class — though at $n = 4\,000$ instead, `LinearSVC` would at least be feasible.

Solution — Extended Exercise 9.3 (2/2)

Solution — brief C, and the common thread

C — an SVM, specifically a one-class SVM. "Almost all traces normal" means the negative class is barely observed, so a two-class classifier is the wrong framing: fit the *support* of the normal class and flag what falls outside. All 60 features are continuous, so scaling is straightforward, and a $12\,000 \times 12\,000$ kernel matrix is large but tractable. Crucially, operations sets the threshold *later*, so a monotone score from `decision_function` is exactly the right deliverable — no probability needed. *What would change my mind:* once enough labelled faults accumulate, a supervised gradient-boosted model would probably overtake it.
The common thread. The SVM wins where p/n is hostile, where the deliverable is a *score* rather than a probability, and where a bespoke similarity is natural. It loses where n is huge, where probabilities are contractual, and where a human must be told why.

Common mistake

Choosing on cross-validated accuracy alone. In all three briefs the binding constraints are computational and institutional; accuracy is the tie-breaker, not the criterion.